

Instituto Federal
Campus Goiânia

Bacharelado em Sistemas de Informação

POO II

Prof. Dory Gonzaga Rodrigues

**Título**

Aula Prática

Persistência de Dados com JDBC



Objetivo

- Revisar o ciclo de vida JDBC
- Implementar operações CRUD (Create, Read, Update, Delete)
- Utilizar PreparedStatement e transações
- Organizar o código em camadas (entidade, acesso a dados, interface)



Estrutura do Projeto

```
src/  
  |— entidades/  
    |   └─ Aluno.java  
  |— jdbc/  
    |   |— DB.java  
    |   └─ AlunoJDBC.java  
  |— telas/  
    |   └─ AlunoTela.java  
  |— programa/  
    |   └─ Principal.java  
propriedade/  
  └─ propriedade.properties  
lib/  
  └─ postgresql-42.x.x.jar
```



Configuração

- Adicionar o .jar do PostgreSQL ao classpath
- Arquivo de propriedades

```
host=jdbc:postgresql://localhost:5432/jdbc  
user=postgres  
Password=aluno
```



Gerenciamento da Conexão

Classe DB.java

- Carrega as configurações de arquivo externo
- Padrão Singleton para a conexão
- Métodos: *getConexao()*, *fechaConexao()*



Entidade

Classe Aluno.java

- Mapeamento da tabela aluno

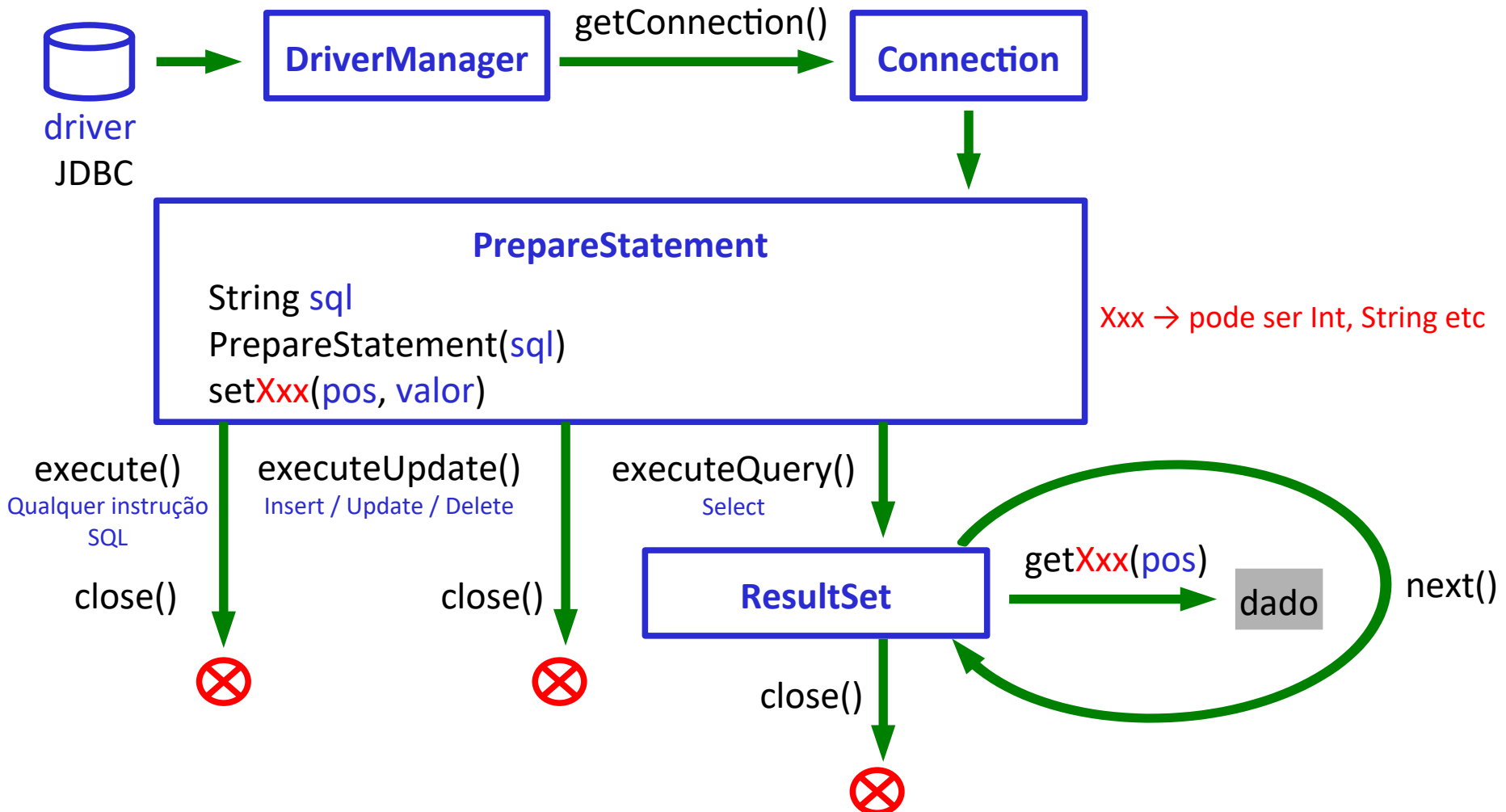
- Atributos:

id, nome, sexo, dt_nasc

- Métodos:

Construtores, getters, setters, toString(), equals(), hashCode()

JDBC

Ciclo de Vida



JDBC

Classe AlunoJDBC.java

Inserir (salvar)

```
public static boolean salvar(Aluno a) throws SQLException {  
    String sql = "INSERT INTO aluno (nome, sexo, dt_nasc) VALUES (?, ?, ?)";  
    try (Connection cnx = DB.getConexao());  
        PreparedStatement pst = cnx.prepareStatement(sql)) {  
        pst.setString(1, a.getNome());  
        pst.setString(2, a.getSexo());  
        pst.setDate(3, a.getDt_nasc());  
        return pst.executeUpdate() >= 1;  
    }  
}
```

JDBC

Classe AlunoJDBC.java

Listar Todos os Alunos

```
public static List<Aluno> listar() throws SQLException {  
    List<Aluno> alunos = new ArrayList<>();  
    String sql = "SELECT * FROM aluno";  
    try (Connection cnx = DB.getConexao();  
        Statement st = cnx.createStatement();  
        ResultSet rs = st.executeQuery(sql)) {  
        while (rs.next()) {  
            alunos.add(new Aluno(  
                rs.getInt("id"),  
                rs.getString("nome"),  
                rs.getString("sexo"),  
                rs.getDate("dt_nasc")  
            ));  
        }  
    }  
    return alunos;  
}
```



JDBC

Classe AlunoJDBC.java

Alterar aluno

```
public static boolean alterar(Aluno a) throws SQLException {  
    String sql = "UPDATE aluno SET nome=?, sexo=?, dt_nasc=? WHERE id=?";  
    try (Connection cnx = DB.getConexao());  
        PreparedStatement pst = cnx.prepareStatement(sql)) {  
        pst.setString(1, a.getNome());  
        pst.setString(2, a.getSexo());  
        pst.setDate(3, a.getDt_nasc());  
        pst.setInt(4, a.getId());  
        return pst.executeUpdate() >= 1;  
    }  
}
```



JDBC

Classe AlunoJDBC.java

Excluir aluno

```
public static boolean apagar(int id) throws SQLException {  
    String sql = "DELETE FROM aluno WHERE id = ?";  
    try (Connection cnx = DB.getConexao();  
        PreparedStatement pst = cnx.prepareStatement(sql)) {  
        pst.setInt(1, id);  
        return pst.executeUpdate() >= 1;  
    }  
}
```



Tratamento de Exceções Específicas

- Capturar SQLException e verificar o SQLState para mensagens amigáveis
- Exemplo: código 23505 (violação de chave única)



Validação de Dados

- Criar método validarAluno(Aluno a) antes de salvar/alterar
- Verificar: nome não vazio, sexo válido, data não futura

Transações

- Desabilitar o AutoCommit
- Realizar os comandos em lote
- Verificar o sucesso da execução
- Habilitar o AutoCommit

```
cnx.setAutoCommit(false);
try {
    // preparar e executar batch
    int[] resultados = pst.executeBatch();
    // verificar sucesso
    cnx.commit();
} catch (SQLException e) {
    cnx.rollback();
}
finally {
    cnx.setAutoCommit(true);
}
```



Interface com o Usuário

Classe AlunoTela.java

- Métodos estáticos para cada tela
- Usa Scanner para entrada
- Chama os métodos da camada JDBC



Classe Principal

- Loop do menu com switch
- Chamada das telas conforme opção
- Bloco finally para fechar conexão



Conclusão

- Prática de todos os passos do ciclo JDBC
- Separação de responsabilidades (camadas)
- Uso de PreparedStatement para segurança e performance
- Controle de transações para operações atômicas